



Lista de Exercícios 1

8 de abril de 2022

Engenharia de Computação

Lilian Giselly Pereira Santos

Pedro Henrique de Brito Nascimento

Projeto e Análise de Algoritmos - Rian Gabriel Pinheiro

Capítulo 1 - Corretude de Algoritmos

1. Faça uma pesquisa (escreva pelo menos 10 linhas) sobre al-Khorezmi (também al-Khwarizmi), o homem de cujo nome deriva a palavra “algoritmo”. Mostre o que as origens das palavras “algoritmo” e “álgebra” têm em comum.

Solução:

Mohamed ibn Musa al-Khwarizmi, nascido por volta do ano 780, viveu em Bagdá, localizada no Iraque. Ele trabalhou na Casa da Sabedoria, um lugar de pesquisas científicas, e lá estudou as obras de sábios árabes, gregos e indianos.

Al-Khwarizmi ficou conhecido por criar novas maneiras de solucionar problemas matemáticos. Dentre os livros escritos por ele, um explicava o método de sistemas de soluções que hoje é conhecido por “álgebra”, palavra cuja origem vem da expressão árabe al-jabr, que aparece no título do livro. Este livro foi mais utilizado nas universidades européias no ensino da matemática.

A palavra algoritmo se origina do árabe “al-khwarizmi” influência do nome do matemático árabe do século IX. Já o termo álgebra, deriva do termo al-jabr, como já citado anteriormente, este termo nomeou o livro produzido pelo matemático onde estava descrito um método para solução de equações do segundo grau, utilizando métodos que somente com o que conhecemos hoje por álgebra e a sua simbologia é possível visualizar, como a representação de incógnitas e raízes de uma equação, bem como a solução de sistemas de equações.

2. Prove por indução que:

- $\sum_{i=1}^n i = \frac{n(n+1)}{2}$, para $n \geq 1$
- $\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$, para $n \geq 1$
- $\sum_{i=1}^n i^3 = \frac{n^2(n+1)^2}{4}$, para $n \geq 1$

Solução:

(a) Desejamos provar que:

$$1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$$

Para isto, utilizaremos a prova por indução e, neste caso:

Passo base ($k = 1$):

Neste caso, $\sum_{i=1}^{i=k} i = \frac{1(1+1)}{2} = 1 \quad \therefore \quad P(1)$ é verdadeiro

Hipótese de Indução ($k \leq n$):

Supondo por hipótese de indução que: $\sum_{i=1}^{i=k} i = \frac{k(k+1)}{2}, \forall k \leq n$

Passo indutivo:

Segue-se para o passo da indução onde, aceitando a hipótese de indução $P(k)$, $k \leq n$, provaremos que se $P(k+1)$ é verdadeiro, então $P(k)$ também é, por indução. Logo:

$$\begin{aligned} \sum_{i=1}^{i=k+1} i &= \underline{1 + 2 + 3 + \dots + k} + (k + 1) = \frac{k(k+1)}{2} + (k + 1) \\ &= \frac{k(k+1) + 2(k+1)}{2} = \frac{(k+1)(k+2)}{2} \end{aligned}$$

$\therefore \quad P(k+1)$ é verdadeiro. $\Rightarrow P(k)$ é verdadeiro



(b) Desejamos provar que:

$$1 + 2^2 + 3^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6}$$

Para isto, utilizaremos a prova por indução e, neste caso:

Passo base ($k = 1$):

Neste caso, $\sum_{i=1}^{i=k} i^2 = \frac{1(1+1)(2 \cdot 1 + 1)}{6} = \frac{6}{6} = 1 \quad \therefore \quad \text{Verdadeiro}$

Hipótese de Indução ($k \leq n$):

Supondo por hipótese de indução que: $\sum_{i=1}^{i=k} i^2 = \frac{k(k+1)(2k+1)}{6}, \forall k \leq n$

Passo indutivo:

Segue-se para o passo da indução onde, aceitando a hipótese de indução $P(k)$, $k \leq n$, provaremos que se $P(k+1)$ é verdadeiro, então $P(k)$ também é, por indução. Logo:

$$\begin{aligned} \sum_{i=1}^{i=k+1} i^2 &= 1 + 2^2 + 3^2 \dots + k^2 + (k+1)^2 = \frac{k(k+1)(2k+1)}{6} + (k+1)^2 \\ &= \frac{k(k+1)(2k+1) + 6(k+1)^2}{6} = (k+1) \frac{k(2k+1) + 6(k+1)}{6} \\ &= (k+1) \frac{(2k^2+k) + (6k+6)}{6} \\ &= (k+1) \frac{2k^2+7k+6}{6} \\ &= \frac{(k+1)(k+2)(2k+3)}{6} = \frac{(k+1)((k+1)+1)(2(k+1)+1)}{6} \end{aligned}$$

$\therefore P(k+1)$ é verdadeiro. $\Rightarrow P(k)$ é verdadeiro

■

(c) Desejamos provar que:

$$1^3 + 2^3 + 3^3 + \dots + n^3 = \frac{n^2(n+1)^2}{4}$$

Para isto, utilizaremos a prova por indução e, neste caso:

Passo base ($k = 1$):

Neste caso, $\sum_{i=1}^{i=k} i^3 = \frac{1^2(1+1)^2}{4} = \frac{4}{4} = 1 \quad \therefore P(1)$ é verdadeiro

Hipótese de Indução ($k \leq n$):

Supondo por hipótese de indução que: $\sum_{i=1}^{i=k} i^3 = \frac{k^2(k+1)^2}{4}, \forall k \leq n$

Passo indutivo:

Segue-se para o passo da indução onde, aceitando a hipótese de indução $P(k)$, $k \leq n$, provaremos que se $P(k+1)$ é verdadeiro, então $P(k)$ também é, por indução. Logo:

$$\begin{aligned} \sum_{i=1}^{i=k+1} i^3 &= 1 + 2^3 + 3^3 \dots + k^3 + (k+1)^3 = \frac{k^2(k+1)^2}{4} + (k+1)^3 \\ &= \frac{k^2(k+1)^2 + 4(k+1)^3}{4} = \frac{(k+1)^2[k^2 + 4(k+1)]}{4} = \frac{(k+1)^2(k^2 + 4k + 4)}{4} \\ &= \frac{(k+1)^2(k+2)^2}{4} = \frac{(k+1)^2((k+1)+1)^2}{4} \end{aligned}$$

$\therefore P(k+1)$ é verdadeiro. $\Rightarrow P(k)$ é verdadeiro

■

3. Prove que:

- $n^3 + 2n$ é divisível por 3 para todo $n \geq 0$, por indução.
- Se $2|3m$ (2 divide $3m$) então $2|m$, por contra-posição.
- A soma de 3 números consecutivos é múltiplo de 3, use prova direta.

Solução:

(a) Desejamos provar que: $n^3 + 2n$ é divisível por 3 para todo $n \geq 0$.

Para isto, utilizaremos a prova por indução e, neste caso:

Passo base ($k = 0$):

Neste caso, $n^3 + 2n = 0^3 + 2 \cdot 0 = 0 \quad \therefore \quad P(1)$ é verdadeiro

Hipótese de Indução ($k \leq n$):

Supondo por hipótese de indução que: 3 divide $k^3 + 2k$

Passo indutivo:

Segue-se para o passo da indução onde, aceitando a hipótese de indução $P(k)$, $k \leq n$, provaremos que se $P(k+1)$ é verdadeiro, então $P(k)$ também é, por indução. Logo:

$$\begin{aligned}(k+1)^3 + 2(k+1) &= (k^3 + 3k^2 + 3k + 1) + 2k + 2 = \\ &= (k^3 + 2k) + (3k^2 + 3k + 1 + 2) = \underline{(k^3 + 2k)} + \underline{3(k^2 + k + 1)}\end{aligned}$$

Por hipótese, temos que $k^3 + 2k$ é divisível por 3. Além disso, através da definição de divisibilidade: “Um número inteiro não nulo a divide um inteiro b , se existe um inteiro c , tal que $b = a \cdot c$ ” podemos afirmar, então, que $3 \cdot (k^2 + k + 1)$ também é divisível por 3. Logo, como ambas as partes são divisíveis por 3, podemos afirmar que $P(k+1)$ também é.

$\therefore \quad P(k+1)$ é verdadeiro. ■

(b) Desejamos provar que se $2|3m$ (2 divide $3m$) então $2|m$. Para isto, utilizaremos a prova por contra-posição, $p \rightarrow q = \neg q \rightarrow \neg p$, onde:

- **p:** $2 | 3m$
- **q:** $2 | m$

Dessa forma, para provarmos $\neg q \rightarrow \neg p$, temos pela definição de divisibilidade:

Hipótese: $m = 2k + 1$, onde k é qualquer número inteiro.

Objetivo: $3m = 2j + 1$, onde j é qualquer número inteiro.

Desenvolvendo:

$$m = 2k + 1 \Rightarrow 3m = 3(2k + 1)$$

$$= 6k + 3 = 2(3k + 1) + 1$$

$$\therefore 3m = 2j + 1$$

Portanto, provamos que $\neg q \rightarrow \neg p$ o que implica, pela prova contrapositiva, que $p \rightarrow q$. Dessa forma, está provado que se $2|3m$ então $2|m$. ■

- (c) Desejamos provar que a soma de 3 números consecutivos é múltiplo de 3. Para isto, utilizaremos prova direta:

Hipótese: $3|a + (a + 1) + (a + 2)$

Prova:

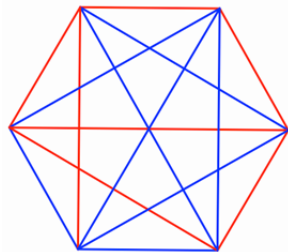
$$a + (a+1) + (a+2) = 3a + 3 = 3(a + 1) = 3k,$$

onde k é qualquer número inteiro.

Pela definição de divisibilidade, temos que se existe um inteiro k , tal que $j = w \cdot k$, então, um número inteiro não nulo w divide um inteiro j , dessa forma, podemos afirmar, então, que $a + (a+1) + (a+2)$ é divisível por 3 e portanto, $a + (a+1) + (a+2)$ é múltiplo de 3. ■

4. Considere um hexágono regular cujos vértices são $v_1, v_2, \dots, v_6$. Mostre que toda maneira de colorir os segmentos de retas que unem dois vértices, utilizando as cores azul ou branca, produz pelo menos um triângulo cujos lados tem a mesma cor.

Solução: Considerando um hexágono regular e suas propriedades básicas, é intuitivo notar que existem inúmeras maneiras de colorir os segmentos de retas que unem dois vértices e na figura abaixo está ilustrada uma delas:

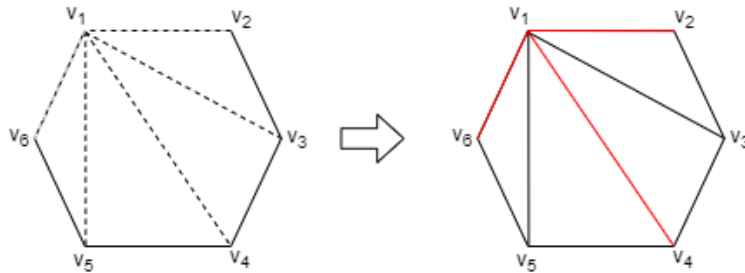


Observe que tomamos a liberdade de mudar a cor branca sugerida no enunciado para a cor vermelha, a fim de promover uma melhor visualização do problema.

Com isso em mente, queremos provar que utilizando apenas duas cores - em nosso caso, azul e vermelho - para colorir os segmentos de retas, teremos pelo menos um triângulo com seus três lados possuindo a mesma cor, independente da forma que escolhermos distribuir as cores.

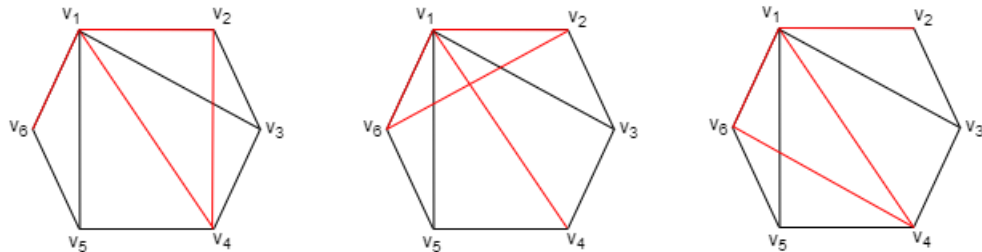
Através de prova direta, faremos a demonstração:

Tomando v_1 como vértice base, sabemos que há 5 segmentos de reta em conexão com ele, assim como mostram as linhas pontilhadas do hexágono à esquerda na figura abaixo. Por termos apenas duas cores, podemos afirmar que pelo menos três destes segmentos possuirão a mesma cor. Em nosso exemplo, escolheremos a cor vermelha e diremos que os vértices v_2 , v_4 e v_6 estão interligados à v_1 através de um segmento vermelho, assim como ilustra a figura à direita.

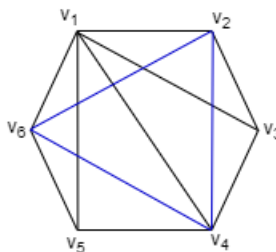


Prosseguindo com a nossa prova:

- Se entre os segmentos $\overline{v_2v_4}$, $\overline{v_2v_6}$ e $\overline{v_4v_6}$, ao menos um deles é vermelho conseguimos formar um triângulo completamente monocromático, em algum dos formatos abaixo, o que nos prova a existência de pelo menos um triângulo de lados de mesma cor.



- No entanto, caso a afirmativa anterior seja falsa, só podemos assumir então que os três segmentos $\overline{v_2v_4}$, $\overline{v_2v_6}$ e $\overline{v_4v_6}$ são azuis. Dessa forma, também provamos a existência de um triângulo monocromático porém, dessa vez, azul.



■

5. Prove a corretude do algoritmo *bubblesort*:

```
1: procedure ALGORITMO BUBBLESORT(vetor  $A[1, \dots, n]$ )
2:   for  $i \leftarrow n - 1 \rightarrow 1$  do
3:     for  $j \leftarrow 0 \rightarrow i - 1$  do
4:       if  $A[j] < A[j + 1]$  then
5:         troca( $A[j], A[j + 1]$ )
6:       end if
7:     end for
8:   end for
9: end procedure
```

Solução:

Para provar a corretude do algoritmo *Bubblesort* é necessário provar os dois invariantes das estruturas de repetição presentes no mesmo. Primeiro, o laço interno e, por fim, a propriedade do primeiro laço servirá de base para a prova do laço externo. Assim, dando continuidade para a prova:

Invariante do laço interno:

Teorema: Depois da iteração j do loop interno, o menor elemento do vetor $A[0, 1, \dots, j]$ estará na posição j .

A prova será feita por indução:

(Inicialização)

Passo Base ($j=0$)

Neste caso, o passo base ocorre antes da iteração do loop, pois no caso onde $j=0$, temos o vetor $A[0, \dots, 0]$, constituído apenas pelo elemento a_0 , desse modo, ele é o menor elemento do vetor e, portanto, $P(j=0)$ é verdadeiro.

(Manutenção)

Hipótese da Indução:

Como hipótese indutiva, vamos assumir primeiramente que após a iteração j , o menor elemento do vetor esteja na posição j .

Passo Indutivo ($j \geq 0$):

Assim, para concluir, é necessário mostrar que após a iteração $j+1$, o menor elemento do vetor $A[0, 1, \dots, j, j+1]$ estará na posição $j+1$.

Sabemos pela hipótese de indução que após a iteração j , o elemento da posição j (a_j) é o menor elemento. Sendo assim, na iteração $j+1$, ou o menor elemento no vetor $A[0, 1, \dots, j, j+1]$ é a_j ou é a_{j+1} .

De todo modo, pela linha (4) o algoritmo verifica e garante que se $a_j < a_{j+1}$, ocorra a troca dos mesmos, de modo a preservar o menor elemento na posição $j+1$ e, assim, fica assegurado que se há $j+1$ posições no vetor $A[0, 1, \dots, j, j+1]$, então, após $j+1$ iterações, o menor elemento estará na posição $j+1$.

(Término)

Portanto, por indução, após j iterações o menor elemento encontra-se na posição j . Em outras palavras, ao final da execução do laço de repetição, a posição j será ocupada pelo menor elemento do vetor $A[0, 1, \dots, j]$.

Invariante do laço externo:

Teorema: Após a iteração “ i ” do loop externo, os “ $n - i$ ” menores elementos do vetor $A[0, 1, \dots, n - 1]$ estarão em ordem decrescente nas posições i até $n-1$, ocupando portanto, o vetor $A[i, \dots, n - 1]$.

(Inicialização)

De fato, na iteração $n - 1$ do loop externo os 1 menores elementos ocupam as posições do vetor $A[n - 1, \dots, n - 1]$, que só possui um elemento nas posições previamente citadas ($n - 1$ até $n - 1$) e conseqüentemente está em ordem decrescente. Portanto, o invariante é válido.

(Manutenção)

O invariante de laço é válido para a inicialização, ou primeira iteração, pois de fato, só há um elemento, conseqüentemente em ordem decrescente, ocupando a posição $n-1$. Logo, resta analisar as próximas “ i ” iterações.

Sendo assim, sabendo que o laço interno garante que o menor elemento do vetor $A[0, \dots, i]$ estará na posição i e que ao fim do laço externo o i é decrementado para $i - 1$. Analogamente, na próxima iteração o subvetor $A[0, \dots, i - 1]$, terá seu menor valor posicionado em $A[i - 1]$ e assim sucessivamente. Desse modo, é fácil observar que após estas iterações, a seguinte proposição é verdadeira:

$$a_{i-1} \geq a_i$$

Logo, é possível concluir também que os elementos comparados até então estarão, com base na linha (4), em ordem decrescente, mantendo a propriedade de ordenação durante a execução do laço de repetição.

(Término)

Após o término do laço de repetição externo, onde $i=1$, é importante lembrar que o laço de repetição interno foi executado, analisando os elementos do vetor $A[0, \dots, i]$, logo $A[0, \dots, 1]$, e conseqüentemente posicionando o elemento de menor valor na posição 1 do vetor. Portanto, considerando que a inicialização e manutenção foram válidas, todos os $n - i$ menores elementos do vetor $A[0, \dots, n - 1]$ estão nas posições 1 até $n - 1$, conseqüentemente o elemento a_0 é o maior elemento do vetor $A[0, \dots, n - 1]$.

Portanto, a seguinte proposição está correta:

$$a_0 \geq a_1 \geq \dots \geq a_{n-2} \geq a_{n-1}$$

Consequentemente, os elementos do vetor $A[0, \dots, n-1]$, estão ordenados de maneira decrescente.



6. Prove a corretude do algoritmo de Horner para a avaliação de polinômios.
 $P(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$.

```
1: procedure ALGORITMO DE HORNER(vetor  $A[0, \dots, n]$ , real  $x$ )
2:    $p \leftarrow A[n]$ 
3:   for  $i \leftarrow n-1 \rightarrow 0$  do
4:      $p \leftarrow p * x + A[i]$ 
5:   end for
6:   return  $p$ 
7: end procedure
```

Solução:

Teorema: O algoritmo de Horner está correto, ou seja, retorna corretamente o cálculo do polinômio para um valor real dado.

Prova:

Considere o seguinte invariante de laço:

$$p = \sum_{j=i}^n A_j x^{j-i}$$

Inicialização: Neste caso, fica garantido, pela linha (2) que, antes de executarmos o laço de iteração propriamente dito, p resulta no coeficiente do termo de maior grau do polinômio informado, isto é: $A[n]$. Assim, considerando que a operação antes do laço de repetição torna $i = n$ (para fins de simplificação), a 2ª linha de código mostra que o invariante de laço está correto antes do *loop for*.

Manutenção: É possível então prosseguir para a execução do laço nos passos seguintes, onde “ i ” é decrementado uma unidade em cada iteração e, de mesmo modo, a linha (4) garante que o valor do próximo termo do invariante será incluído na soma, assim como o valor do termo atual. Sendo que o resultado

da operação do termo atual pelo valor real x , informado pelo usuário, vai sendo parcialmente computado em cada laço de iteração. Desse modo, para:

$$i = n - 1 : \quad \sum_{j = n-1}^n A_j \, x^{j-i} = A_{n-1} x^0 + A_n x^1$$

$$i = n - 2 : \quad \sum_{j = n-2}^n A_j x^{j-i} = A_{n-2}x^0 + A_{n-1}x^1 + A_nx^2$$

• • • • •

$$i = 1 : \quad \sum_{j=1}^n A_j x^{j-i} = A_1 x^0 + A_2 x^1 + \dots + A_{N-1} x^{N-2} + A_N x^{N-1}$$

Sendo assim, ao longo da execução dos laços de repetição

$$p = \sum_{j=1}^n A_j x^{j-i}$$

Término: Sabendo que a variável “ i ” está sendo decrementada e avaliada de $n - 1$ até 0, e considerando que a inicialização e manutenção do laço de repetição são válidas. Após a execução da última iteração “ $i = 0$ ”, temos que o valor de p é:

$$\begin{aligned} \sum_{j=0}^n A_j x^{j-i} &= A_0 x^0 + A_1 x^{1-0} + \dots + A_{N-1} x^{(N-1)-0} + A_N x^{N-0} \\ &= A_0 x^0 + A_1 x^1 + \dots + A_{N-1} x^{N-1} + A_N x^N \end{aligned}$$

Portanto, conclui-se que o algoritmo de Horner está correto e é capaz de calcular o valor do polinômio de Grau N dado para um determinado valor de x.



7. Prove a corretude do algoritmo Conversor Decimal-Binário.

```
1: procedure CONVERSOR D-B(inteiro  $n$ )
2:    $t \leftarrow n$ 
3:    $k \leftarrow 0$ 
4:   zere todos os bits de  $b$ 
5:   while  $t > 0$  do
6:      $k \leftarrow k + 1$ 
7:      $b[k] \leftarrow t \bmod 2$ 
8:      $t \leftarrow t \div 2$ 
9:   end while
10:  return  $b$ 
11: end procedure
```

Solução:

Teorema: O algoritmo Conversor Decimal-Binário está correto, ou seja, retorna corretamente a conversão em binário para um número natural dado.

Prova:

Sendo:

- m_k é o inteiro que representa o estado atual do vetor b , após k iterações, tal que:

$$m_k = \begin{cases} 0, & \text{se } k = 0 \\ \sum_{i=1}^k b[i] 2^{i-1}, & \text{se } k \geq 1 \end{cases}$$

- t_k representa o valor de t ao final do k -ésimo loop.

Considere o seguinte invariante de laço, onde na iteração k , o vetor $b[1...k]$ representa um inteiro m_k tal que:

$$n(k) = m_k + t_k \cdot 2^k, \forall k$$

Inicialização: Neste caso, a variável t é inicializada com o número natural a ser convertido n , o contador $k = 0$ e o vetor b está vazio. Além disso, a iteração na inicialização é:

$$\text{para } k = 0: \quad n(0) = t_0 \cdot 2^0 + m_0 = n \cdot 1 + 0 \therefore n(0) = n$$

Dessa forma, a definição de invariante na inicialização está satisfeita.

Manutenção: Prosseguimos então para a execução do laço nos passos seguintes, onde k é incrementado uma unidade a cada iteração e, dessa forma, o vetor b na posição k é atualizado com o valor da operação modular com 2, isto é, $t_k \bmod 2$. Dando prosseguimento com a atualização do valor de t_k ao ser dividido por 2, o que permite que haja o algoritmo continue para que a conversão pare quando o número for menor ou igual a zero. Dessa forma, precisamos mostrar que se o invariante de laço é válido para k , ele será válido para $k+1$. Supomos, então, que $n(k)$ é válido, então sabemos que $n(k) = t_k \cdot 2^k + m_k$ e a partir do corpo do loop deduzimos que:

$$\begin{aligned} k' &= k + 1 \\ b[k'] &= t_k \bmod 2 \\ t_{k'} &= t_k \div 2 \end{aligned}$$

Assim, temos dois casos a provar:

- Quando t_k é par, então $b[k'] = 0$ e $b[1...k']$ ainda representa m_k .

$$\therefore m_{k'} = m_k$$

$$\begin{aligned} \text{Logo } n(k) &= m_k + t_k \cdot 2^k \Rightarrow n(k') = m_{k'} + t_{k'} \cdot 2^{k'} \\ &= m_k + (t_k/2) \cdot 2^{k'} = m_k + t_k \cdot 2^k = n \end{aligned}$$

, temos então que $n(k+1)$ é válido $\forall t_k$ par (I)

- Quando t_k é ímpar, isso nos dá então que $b[k'] = 1$ e $b[1...k']$ representa $2^k + m_k$.

$$\begin{aligned} \text{Pois, } m_k &= \sum_{i=1}^{i=k} b[i] \cdot 2^{i-1} \Rightarrow m_{k+1} = \sum_{i=1}^{i=k+1} b[i] \cdot 2^{i-1} = \sum_{i=1}^{i=k'} b[i] \cdot 2^{i-1} \\ &= \sum_{i=1}^{i=k'} b[i] \cdot 2^{i-1} = \underline{b[k']} \cdot 2^k + \sum_{i=1}^{i=k} b[i] \cdot 2^{i-1} \\ \therefore m_{k'} &= 2^k + m_k \end{aligned}$$

Sabendo disso e que, quando t_k é ímpar $t_{k'} = \frac{t_k-1}{2}$, segue-se que:

$$\begin{aligned} n(k) &= m_k + t_k \cdot 2^k \Rightarrow n(k') = m_{k'} + t_{k'} \cdot 2^{k'} \\ &= (m_k + 2^k) + \frac{t_k - 1}{2} \cdot 2^{k'} \\ &= (m_k + 2^k) + (t_k - 1) \cdot 2^k \\ &= m_k + 2^k + t_k \cdot 2^k - 2^k = m_k + t_k \cdot 2^k = n \end{aligned}$$

Temos então que $n(k + 1)$ é válido $\forall t_k$ ímpar. (II)

Logo, de (I) e (II), temos que o invariante é válido $\forall t_k$

Término: Sabemos que o laço de repetição executa enquanto o nosso inteiro t é maior do que zero, no entanto, isso acontece considerando que t inicializa com n e é decrementado dividindo seu valor por dois a cada interação. Desse modo, na última interação, quando “ $t \leq 0$ ” e “ $k = n$ ”, o valor de n é:

$$n = t_n \cdot 2^n + m_n$$

Portanto, conclui-se que o algoritmo Conversor D-B está correto e, de fato, converte para binário um número natural dado. ■

8. A sequência de Fibonacci é definida da seguinte forma: $f_0 = 0$; $f_1 = 1$ e $f_{i+2} = f_{i+1} + f_i$, $\forall i \geq 0$. Prove que para todo $n \geq 1$ temos:

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n = \begin{pmatrix} f_{n+1} & f_n \\ f_n & f_{n-1} \end{pmatrix},$$

em que o lado esquerdo representa n -ésima potência de uma matriz 2×2 .

Solução:

A prova pode ser feita por indução:

Passo Base: ($n = 1$)

$$A^n = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^1 = \begin{pmatrix} f_{1+1} & f_1 \\ f_1 & f_0 \end{pmatrix} = \begin{pmatrix} f_2 & f_1 \\ f_1 & f_0 \end{pmatrix}$$

Considerando a sequência de Fibonacci até os 3 primeiros termos, temos que:

$$f_0 = 0 \qquad f_1 = 1 \qquad f_2 = f_0 + f_1 = 1$$

e, portanto, a matriz $A = \begin{pmatrix} f_2 & f_1 \\ f_1 & f_0 \end{pmatrix}^1 = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$, associa direta e corretamente a sequência de Fibonacci para os seus 3 primeiros elementos.

Hipótese de Indução: ($n \leq k$)

Suponha que a hipótese de indução $A^k = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^k = \begin{pmatrix} f_{k+1} & f_k \\ f_k & f_{k-1} \end{pmatrix}$, $\forall n \leq k$.

Passo Indutivo: ($n = k + 1$)

Deseja-se saber se a hipótese é válida $\forall n > k$. Portanto, sabendo que

$$A^{k+1} = A^k \cdot A^1, \text{ por definição, e } A^k = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^k, \text{ por hipótese.}$$

Segue-se que:

$$\begin{aligned} A^k \cdot A &= \begin{pmatrix} f_{k+1} & f_k \\ f_k & f_{k-1} \end{pmatrix} \cdot \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} f_{k+1} + f_k & f_{k+1} \\ f_k + f_{k-1} & f_k \end{pmatrix} \\ \therefore A^k \cdot A &= \begin{pmatrix} f_{k+2} & f_{k+1} \\ f_{k+1} & f_k \end{pmatrix} \Rightarrow A^{k+1} = \begin{pmatrix} f_{k+2} & f_{k+1} \\ f_{k+1} & f_k \end{pmatrix} \end{aligned}$$

Portanto, por indução, se a hipótese é válida para $n = k$ então é válida para $n = k + 1$, logo a matriz $A^n = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n$ provê os $\{n + 1, n, n - 1\}$ -ésimos termos da sequência de Fibonacci, $\forall n \geq 1$.

■

9. O BIN PACKING é um problema cuja entrada consiste em: n itens com tamanhos $s_1, s_2, \dots, s_n$ em que $s_i \in [0, 1]$. O objetivo é encontrar o menor número de “bins” (caixas) unitárias para armazenar os n itens. Dado os algoritmos a seguir, mostre que eles não encontram a solução ótima do problema, ou seja, encontre contraexemplos para cada um dos seguinte algoritmos para o problema.

- (A) Coloque na *bins* os elementos em ordem da esquerda para a direita, se ele couber, caso contrário tente na próxima *bin*;
- (B) Coloque na *bin* mais livre o maior elemento;
- (C) Coloque o menor elemento na *bin* mais livre.

Solução: Supondo que cada *bin* consiga armazenar o equivalente a um item de tamanho 1, temos que:

- (a) Caso de Entrada: (0.2, 0.8, 0.5, 0.5, 0.7, 1, 0.3)
Saída do Algoritmo **A**: [0.2, 0.8], [0.5, 0.5], [0.7], [1], [0.3]
Saída Otimizada: [0.2, 0.8], [0.5, 0.5], [0.7, 0.3], [1]
- (b) Caso de Entrada: (0.2, 0.8, 0.5, 0.5, 0.7, 1, 0.3)
Saída do Algoritmo **B**: [1], [0.8], [0.7], [0.5, 0.5], [0.3, 0.2]
Saída Otimizada: [0.2, 0.8], [0.5, 0.5], [0.7, 0.3], [1]
- (c) Caso de Entrada: (0.2, 0.8, 0.5, 0.5, 0.7, 1, 0.3)
Saída do Algoritmo **C**: [0.2, 0.3, 0.5], [0.5], [0.7], [0.8], [1]
Saída Otimizada: [0.2, 0.8], [0.5, 0.5], [0.7, 0.3], [1]

10. Considere o algoritmo de Ulam, ele termina? De fato, conjectura-se que seguindo o algoritmo, sempre será obtida a sequência 4, 2, 1 (Conjectura de Collatz). Ex: Para o valor $a = 22$, será obtida a seguinte sequência: 22, 11, 34, 17, 52, 26, 13, 40, 20, 10, 5, 16, 8, 4, 2, 1. Como a prova do término do algoritmo consiste em um difícil problema matemático em aberto. Implemente um teste exaustivo mostrando que para qualquer unsigned short int (1 a 65535) de entrada o algoritmo para. Escreva um pequeno relato informando o tamanho da maior sequência encontrada, o valor na qual a maior sequência foi obtida, a média dos tamanhos das sequências e o tempo de execução.

```
1: procedure ALGORITMO DE ULAM(inteiro positivo  $a$ )
2:    $x \leftarrow a$ 
3:   while Os três últimos valores de  $x$  não for 4,2,1 do
4:     if  $x$  for par then
5:        $x \leftarrow x/2$ 
6:     else
7:        $x \leftarrow 3x + 1$ 
8:     end if
9:   end while
10: end procedure
```

Solução: Após a implementação do algoritmo e da execução de um teste exaustivo contendo 65535 entradas, ou seja, cobrindo a possibilidade de entrada de qualquer unsigned short int, pudemos observar que, de fato, o algoritmo para de executar e é, portanto, finito. Além disso, assim como ilustra a figura abaixo, algumas informações foram coletadas:

- O tamanho da maior sequência equivale a 340 números;
- O valor de N para a maior sequência é 52527;
- A média dos tamanhos das sequências obtidas é 104,21 números;
- O tempo de execução do algoritmo é 0.06200 segundos.

```

#include<stdio.h>
#include<time.h>
int maior_num=0;
double media_tam=0;
int maior_seq=0;
void ulam(int n){
    int x = n;
    int a3=0,a2=0,a1=0;
    int num_maior_seq=1;
    while((a3!=1) || (a2!=2) || (a1!=4)){
        if(x%2==0) x = x/2;
        else x = 3*x +1;
        a1 = a2;
        a2 = a3;
        a3 = x;
        num_maior_seq++;
        if(num_maior_seq >= maior_seq)
        {
            maior_num = n;
            maior_seq = num_maior_seq;
        }
    }
    media_tam+=num_maior_seq;
}
int main(){
    int n,numeros=0;
    clock_t ini = clock();

    while(scanf("%d", &n)!=EOF){
        ulam(n);
        numeros++;
    }
    media_tam/=numeros;
    clock_t fim = clock();
    double tempo = (double)(fim - ini)/ (double)CLOCKS_PER_SEC;
    printf("Tamanho da Maior Sequencia -> %d numeros\n", maior_seq);
    printf("Valor de N para a maior sequencia -> %d\n",maior_num);
    printf("Media dos Tamanhos das Sequencias -> %.2lf numeros\n", media_tam);
    printf("Tempo de Execucao -> %.5lf segundo(s)", tempo);
}

```

Tamanho da Maior Sequencia -> 340 numeros
 Valor de N para a maior sequencia -> 52527
 Media dos Tamanhos das Sequencias -> 104.21 numeros
 Tempo de Execucao -> 0.06200 segundo(s)